

Napredni modeli i baze podataka

Predavanja
listopad 2025.

Napredni SQL



Napredni SQL - teme

- 1. Pivotiranje**
- 2. Funkcije za rad s podacima u prozoru**
- 3. Rekurzivni upiti i CTE (Common Table Expression)**



Napredni SQL - teme

- 1. Pivotiranje**
2. Funkcije za rad s podacima u prozoru
3. Rekurzivni upiti i CTE (Common Table Expression)

Motivacijski primjer

Odredi broj izlazaka na ispit po godinama i mjesecima u sljedećem obliku:

godina	siječanj	veljača	ožujak	travanj	svibanj	lipanj	srpanj	kolovoz	rujan	listopad	studeni	prosinac
2016	17215	2908	846	728	680	9081	10066	...	...	...	...	...
2017	20400	717	235	196	205	9887	9255	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...

ispit

sifPred	sifStud	datumRok	ocjena
1	1	01.02.2016	1
1	3	01.02.2016	5
1	1	01.03.2016	4
1	2	01.02.2016	5
...	...	...	...

Ovo znamo, ali to nije rješenje zadatka:

```
SELECT EXTRACT(YEAR FROM datumRok) AS godina
      , EXTRACT(MONTH FROM datumRok) AS mjesec
      , COUNT(*) AS brIspita
FROM ispit
GROUP BY godina, mjesec
```

godina	mjesec	brlspita
2016	1	17215
2016	2	2908
...	...	...

Znamo li bolje?

Pivotiranje – bolje rješenje?

```
SELECT DISTINCT EXTRACT(YEAR FROM datumRok) AS godina
, (SELECT COUNT(*)
   FROM ispit i
   WHERE EXTRACT(MONTH FROM i.datumRok) = 1
        AND EXTRACT(YEAR FROM i.datumRok) = EXTRACT(YEAR FROM ispit.datumRok))
, ...
, (SELECT COUNT(*)
   FROM ispit i
   WHERE EXTRACT(MONTH FROM i.datumRok) = 12
        AND EXTRACT(YEAR FROM i.datumRok) = EXTRACT(YEAR FROM ispit.datumRok))
FROM ispit
ORDER BY godina
```

godina	siječanj	veljača	ožujak	travanj	svibanj	lipanj	srpanj	kolovoz	rujan	listopad	studen	prosinac
2016	17215	2908	846	728	680	9081	10066	...	...	...	...	...
2017	20400	717	235	196	205	9887	9255	...	...	...	...	...
...	...	...	...	...	...	...	...	...	...	...	...	...

Jesmo li zadovoljni s rješenjem?

Nespretno i neprimjenjivo u općenitoj situaciji kada je broj stupaca nepoznat.

-

godina	mjesec	brlspita
2016	1	17215
2016	2	2908
...	...	...

	siječanj	veljača	ožujak	travanj	svibanj	lipanj	srpanj	kolovoz	rujan	listopad	studeni	prosinac
2016	17215	2908	846	728	680	9081	10066	...	...	...	...	...
2017	20400	717	235	196	205	9887	9255	...	...	...	...	...
...	...	...		...	...	...	...	...	...	...		

- | | 1 | 2 | 3 | 4 | 5 | Grand Total |
|-------------|---|---|---|---|---|-------------|
| 1.6.2013 | 1 | 1 | 1 | 1 | 1 | 5 |
| 1.7.2013 | 2 | 2 | 2 | 2 | 2 | 10 |
| Grand Total | 3 | 3 | 3 | 3 | 3 | 15 |

Pivotiranje u SQL-u

- Nije definirano SQL standardom
- Podržano u
 - Oracle 11g
 - SQL Server 2005⁺
 - PIVOT
 - PostgreSQL 8.3⁺
 - crosstab funkcija

	Rezultat	
crosstab (text sql)	skup n-torki	Proizvodi "pivot tablicu" koja sadrži imena redaka i N imena stupaca. N (tip i broj vrijednosti) je određen upitom.
crosstab (text srcSql, text categorySql)	skup n-torki	Proizvodi "pivot tablicu". Stupci pivot tablice su specificirani drugim upitom (argumentom).

PostgreSQL: crosstab funkcija

crosstab (text SQL)

Rezultat SQL naredbe u obliku:

prvaVrijURedu	kategorija	vrijednost
red1	kat1	vrij11
red1	kat2	vrij12
red1	kat3	vrij13
	...	...
red2	kat1	vrij21
red2	kat2	vrij22
red2	kat3	vrij23
		...

crosstab funkcija pretvara u sljedeći oblik

prvaVrijURedu	katLabel1	katLabel2	katLabel3	...
red1	vrij11	vrij12	vrij13	...
red2	vrij21	vrij22	vrij23	...
...	...	...	...	...

SQL naredba - argument crosstab funkcije mora biti napisana tako da:

- **vraća trojke** s tim da prva vrijednost predstavlja prvu vrijednost u retku, druga kategoriju stupca a treća je vrijednost ćelije.
- **rezultat bude sortiran** prema *prvaVrijURedu*, a zatim prema *kategorija*.

crosstab funkcija:

- vraća skup n-torki
- imena stupaca **moraju biti eksplicitno definirana** u FROM dijelu SQL naredbe koja ju poziva

```
SELECT * FROM crosstab('...') AS ct (prvaVrijURedu dataType,  
                                     katLabel1      dataType,  
                                     katLabel2      dataType,...) ;
```

Tipovi podataka
od drugog do
zadnjeg moraju
biti jednaki

PostgreSQL: crosstab funkcija - primjer

Odredi broj izlazaka na ispit po godinama i mjesecima u sljedećem obliku:

godina	siječanj	veljača	ožujak	travanj	svibanj	lipanj	srpanj					
2016	17215	2908	846	728	680	9081	10066	...	...	...	...	...
2017	20400	717	235	196	205	9887	9255	...	...	...	...	...
...	...	...		...	...	...	...	...	...	...		

```
SELECT * FROM crosstab(text SQLStatement)
      AS pivotTable (godina INT, siječanj INT, veljača INT,...)
```

Rezultat izvođenja SQLStatement mora biti skup trojki s tim da

1. vrijednost predstavlja godinu
2. vrijednost predstavlja redni broj mjeseca u godini
3. vrijednost predstavlja ukupan broj ispita u godini i mjesecu

godina	mjesec	brlspita
2016	1	17215
2016	2	2908
2016	3	846
...	...	...

```
SELECT EXTRACT(YEAR FROM datumRok) AS godina
      , EXTRACT(MONTH FROM datumRok) AS mjesec
      , COUNT(*) AS brIspita
FROM ispit
GROUP BY EXTRACT(YEAR FROM datumRok), EXTRACT(MONTH FROM datumRok)
ORDER BY EXTRACT(YEAR FROM datumRok), EXTRACT(MONTH FROM datumRok)
```

Rezultat gornje SQL naredbe će crosstab funkcija transformirati u pivot tablicu.

PostgreSQL: crosstab funkcija - primjer

Odredi broj izlazaka na ispit po godinama i mjesecima u sljedećem obliku:

godina	siječanj	veljača	ožujak	travanj	svibanj	lipanj	srpanj					
2016	17215	2908	846	728	680	9081	10066	...	...	...	...	...
2017	20400	717	235	196	205	9887	9255	...	...	...	...	...
...	...	...		...	...	...	...	...	...	...		

```
CREATE EXTENSION tablefunc;
```

```
SELECT *
```

```
FROM crosstab ('SELECT EXTRACT(YEAR FROM datumRok) AS godina
                  , EXTRACT(MONTH FROM datumRok) AS mjesec
                  , COUNT(*) AS brIsp
```

mora biti int

```
FROM ispit
```

```
GROUP BY godina, mjesec
```

```
ORDER BY godina, mjesec')
```

zapisi moraju biti sortirani

```
AS pivotTable (godina INT
```

```
  , siječanj INT, veljača INT, ožujak INT, travanj INT, svibanj INT, lipanj INT
  , srpanj INT, kolovoz INT, rujan INT, listopad INT, studeni INT, prosinac INT)
```

```
ORDER BY godina
```

Preporuka: osigurati odgovarajući tip podatka u skladu s definicijom rezultata SQL naredbe

```
SELECT *
```

```
FROM crosstab ('SELECT EXTRACT(YEAR FROM datumRok)::INT AS godina
                  , EXTRACT(MONTH FROM datumRok)::INT AS mjesec
                  , COUNT(*)::INT AS brIsp
FROM ispit
```

```
...
```

PostgreSQL: crosstab funkcija – primjer

Što ako za neku godinu nije bilo ispita u svakom od 12 mjeseci?

godina	mjesec	brlSpita
2016	12	567
2017	2	17215
2017	5	2908
2018	1	9055
...	...	...

crosstab (text SQL)



	siječanj	veljača	ožujak	travanj	svibanj								
2016	...	...	...	...	...	...	...	...	...	...	...	...	...
2017	17215	2908											
...	...	...	...	...	...	...	...	...	...	...	...	...	...

Krivo! 17215 treba biti u stupcu za veljaču, a 2908 u stupcu za svibanj.

Rješenje je reći koje kategorije postoje kako bi vrijednosti bile prikazane u ispravnim stupcima (stupcima koji se odnose na ispravnu kategoriju - categorySql).

crosstab (text srcSql, text categorySql)

Kod ove inačice crosstab funkcije srcSql može vratiti više od 3 vrijednosti u retku s tim da **prva i zadnje dvije** vrijednosti moraju biti kao kod osnovne inačice, a između se mogu pojaviti neke dodatne vrijednosti:

prvaVrijURedu	ekstraVrijednost1	ekstraVrijednost2	...	kategorija	vrijednost
red1	ekstra1	ekstra2		kat1	vrij11
...	...				...

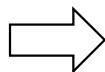
Želim ovakav izvještaj:

akGodina	početak	kraj	siječanj	veljača	ožujak	travanj	svibanj							
2016/2017	01.10.2016	30.09.2017		...	...	...	...	...	...	...	...	...	...	...
2017/2018	01.10.2017	30.09.2018		17215			2908							

PostgreSQL: crosstab funkcija – primjer (poboljšano rješenje)

crosstab (text srcSql, text categorySql)

akGodina	početak	kraj	mjesec	brlspita
2016/2017	01.10.2016	30.09.2017	12	567
2017/2018	01.10.2017	30.09.2018	2	17215
2017/2018	01.10.2017	30.09.2018	5	2908
2018/2019	01.10.2018	30.09.2019	1	9055
...	...	...	...	...



akGodina	početak	kraj	siječanj	veljača	ožujak	travanj	svibanj			
2016/2017	01.10.2016	30.09.2017	...	...	...	...	...	...	...	...
2017/2018	01.10.2017	30.09.2018		17215			2908			
...			...	...	...	...	...	...	...	...

```
CREATE TEMP TABLE mjesec
```

```
(rbrMjesec int);
```

```
INSERT INTO mjesec VALUES (1), (2), (3), (4), (5), (6), (7), (8), (9), (10), (11), (12);
```

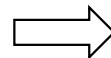
```
SELECT *
FROM crosstab ($$SELECT (EXTRACT(YEAR FROM (datumRok + '3 months'::INTERVAL))-1)::VARCHAR || '/' ||
    EXTRACT(YEAR FROM (datumRok + '3 months'::INTERVAL)) AS AKgodina
    , ('01.10.' || EXTRACT(YEAR FROM (datumRok + '3 MON'::INTERVAL))-1) pocetak
    , ('30.09.' || EXTRACT(YEAR FROM (datumRok + '3 MON'::INTERVAL))) kraj
    , EXTRACT(MONTH FROM datumRok)::INT AS mjesec
    , COUNT(*)::INT AS brIsp
FROM ispit
GROUP BY akGodina, pocetak, kraj, mjesec
ORDER BY akGodina, mjesec$$
, $$SELECT rbrMjesec FROM mjesec ORDER BY rbrMjesec$$)

AS pivotTable (akGodina VARCHAR, početak DATE, kraj DATE
    , siječanj INT, veljača INT, ožujak INT, travanj INT, svibanj INT, lipanj INT
    , srpanj INT, kolovoz INT, rujan INT, listopad INT, studeni INT, prosinac INT)
ORDER BY akGodina
```

PostgreSQL: Pivotiranje – unaprijed nepoznat broj stupaca?

kupacProizvod

kupac	proizvod	količina
Ana	Mlijeko	45
Pero	Sol	45
Mia	Šećer	35
Ivo	Mlijeko	37
Ana	Brašno	65
Pero	Mlijeko	75
Mia	Mlijeko	35
Ivo	Sol	45
...	...	...



kupac	brašno	mlijeko	sol	šećer	...
Ana	65	45			
Ivo			45		
Mia		35		35	
Pero		75	45		

```
SELECT *
FROM crosstab ('SELECT kupac, proizvod, SUM(kolicina)
                FROM kupacProizvod
                GROUP BY kupac, proizvod
                ORDER BY kupac, proizvod'
, 'SELECT DISTINCT proizvod
    FROM kupacProizvod
    ORDER BY proizvod')
AS pivotTable (kupac char(15), Brašno INT, Mlijeko INT, Sol INT, Šećer INT)
ORDER BY kupac
```

generirati dinamički
(Dynamic SQL)

- dinamičko kreiranje i izvođenje SQL naredbu moguće samo u PL/pgSQL - SQL Procedural Language
- Dynamic SQL nije predviđen kao tema u okviru predmeta



Napredni SQL - teme

1. Pivotiranje
- 2. Funkcije za rad s podacima u prozoru**
3. Rekurzivni upiti i CTE (Common Table Expression)

Motivacijski primjer 1

Za predmet i ispitni rok odrediti kumulativan broj ispita i kumulativan prosjek ocjena. Kumulativan broj ispita, odnosno prosjek ocjena, uključuje sve ispite s tog i prethodnih ispitnih rokova iz istog predmeta.

ispit

sifPred	sifStud	datumRok	ocjena
1	1	01.06.2020	1
1	3	01.06.2020	5
1	1	01.07.2020	4
1	2	01.07.2020	5
1	4	01.07.2020	4
1	5	01.07.2020	5
2	1	01.07.2020	3
2	2	01.07.2020	2

sifPred	datumRok	kumBrispit	kumPrOcj
1	01.06.2020	2	3.00
1	01.07.2020	6	4.00
2	01.07.2020	2	2.50

Ovo znamo, ali to nije rješenje zadatka:

```
SELECT sifPred, datumRok
      , COUNT(*)    brIspit
      , AVG(ocjena) prOcj
FROM ispit
GROUP BY sifPred, datumRok
```

sifP	datumRok	brIspit	prOcj
1	01.06.2020	2	3.00
1	01.07.2020	4	4.50
2	01.07.2020	2	2.50

I ovo znamo, i to JE rješenje zadatka:

```
SELECT DISTINCT sifPred, datumRok
      , (SELECT COUNT(*) FROM ispit i
          WHERE ispit.sifPred = i.sifPred
            AND i.datumRok <= ispit.datumRok) kumBrIspit
      , (SELECT AVG(ocjena) FROM ispit i
          WHERE ispit.sifPred = i.sifPred
            AND i.datumRok <= ispit.datumRok) kumPrOjc
FROM ispit
```

Ali,...

2 korelirana podupita ...

Motivacijski primjer 2

Odrediti ukupan broj bodova i rang studenta u okviru predmeta. Rang treba odrediti temeljem ukupnog broja bodova.

studProvjera

sifPred	sifStud	kratProvjera	bod
1	1	1DZ	5.00
1	2	1DZ	10.00
1	3	1DZ	5.00
1	1	MI	10.00
1	2	MI	5.00
1	3	MI	20.00
...	...	...	...
2	1	MI	20.00
2	2	MI	25.00
2	3	MI	30.00

Ovo znamo, ali to nije rješenje zadatka:

```
SELECT sifPred
      , sifStud
      , SUM(bod)      ukBod
FROM studProvjera
GROUP BY sifPred, sifStud
--ORDER BY sifPred, ukBod DESC
```

sifPred	sifStud	ukBod	rang
1	3	25.00	1
1	1	15.00	2
1	2	15.00	2
2	3	30.00	1
2	2	25.00	2
2	1	20.00	3

sifPred	sifStud	ukBod
1	3	25.00
1	1	15.00
1	2	15.00
2	3	30.00
2	2	25.00
2	1	20.00

Motivacijski primjer 3

Odrediti ukupan broj bodova i rang studenta za svaki predmet. Dodatno, odrediti prosjek ukupnog broja bodova koje studenti osvoje na predmetu.

studProvjera

sifPred	sifStud	kratProvjera	bod
1	1	1DZ	5.00
1	2	1DZ	10.00
1	3	1DZ	5.00
1	1	MI	10.00
1	2	MI	5.00
1	3	MI	20.00
...	...	...	...
2	1	MI	20.00
2	2	MI	25.00
2	3	MI	30.00

sifPred	sifStud	ukBod	rang	prosUkBod
1	3	25.00	1	18.333
1	1	15.00	2	18.333
1	2	15.00	2	18.333
2	3	30.00	1	25.000
2	2	25.00	2	25.000
2	1	20.00	3	25.000

Ukupan broj bodova studenta na predmetu:

Prosjek ukupnog broja bodova studenata na predmetu?

sintaksa ne dozvoljava

SUM(bod)

AVG(SUM(bod))

GROUP BY sifPred, sifStud

GROUP BY sifPred

```
SELECT sifPred, sifStud
      , SUM(bod) ukBod
FROM studProvjera
GROUP BY sifPred, sifStud
```

```
SELECT sifPred...
      , AVG(SUM(bod)) prosUkBod
FROM studProvjera
GROUP BY ????
```

Što nam treba, a standardni SQL ne podržava?

- Mogućnost podjele n-torki relacije na prozore koji
 - se određuju dinamički obzirom na okolinu trenutne ntorke
- Mogućnost računanja agregatnih vrijednosti na razini prozora
 - u istom SQL upitu različite agregatne funkcije izračunati temeljem različito definiranih prozora
 - izbjeći „gubitak” n-torke koji se nužno dogodi pri GROUP BY
- Dodatne funkcije čiji rezultat se evaluira temeljem n-torki sadržanih u prozoru (npr. rang)

Prozori i funkcije za rad s prozorima

- Definirane standardom ISO SQL:2003, detaljnije razrađene standardom ISO SQL:2008
- Podržane u Oracle, SQL Server, Sybase, DB2, **PostgreSQL**, Firebird
- Primjenjuju se najčešće u OLAP ali i u OLTP sustavima za upite koji uključuju
 - rangiranje
 - ugniježdene, kumulativne i "pomične" agregacije
 - ...
- Sintaksa i primjeri koji slijede: PostgreSQL

SQL agregatna funkcija: GROUP BY ↔ nad prozorom

SQL agregatna funkcija:

```
SELECT sifPred
      , datumRok
      , COUNT(*)      brIspit
      , AVG(ocjena)   prOcj
FROM ispit
GROUP BY sifPred, datumRok
```

ispit

sif Pred	sif Stud	datumRok	ocjena
1	1	01.06.2020	1
1	3	01.06.2020	5
1	1	01.07.2020	4
1	2	01.07.2020	5
1	4	01.07.2020	4
1	5	01.07.2020	5
2	1	01.07.2020	3
2	2	01.07.2020	2

sif Pred	datumRok	br Ispit	pr Ocj
1	01.06.2020	2	3.00
1	01.07.2020	4	4.50
2	01.07.2020	2	2.50

Funkcija nad podacima u prozoru:

```
SELECT sifPred
      , datumRok
      , COUNT(*) OVER (PARTITION BY
                        sifPred, datumRok) brIsp
      , AVG(ocjena) OVER (PARTITION BY
                           sifPred, datumRok) prOcj
FROM ispit
```

ispit

sif Pred	sif Stud	datumRok	ocjena
1	1	01.06.2020	1
1	3	01.06.2020	5
1	1	01.07.2020	4
1	2	01.07.2020	5
1	4	01.07.2020	4
1	5	01.07.2020	5
2	1	01.07.2020	3
2	2	01.07.2020	2

sif Pred	datumRok	br Isp	pr Ocj
1	01.06.2020	2	3.00
1	01.06.2020	2	3.00
1	01.07.2020	4	4.50
1	01.07.2020	4	4.50
1	01.07.2020	4	4.50
1	01.07.2020	4	4.50
2	01.07.2020	2	2.50
2	01.07.2020	2	2.50

GROUP BY + agregatne funkcije ↔

Funkcije za rad s prozorima

GROUP BY $atr_1, \dots, atr_n$	Funkcije za rad s prozorima
Samo jedan način grupiranja u jednoj SELECT naredbi	Agregati u istoj SELECT naredbi mogu biti izračunati temeljem n-torki sadržanih u različitim prozorima
Vrijednosti agregatnih funkcija se računaju za cijelu grupu	Vrijednosti agregatnih funkcija se računaju za prozore
Jedna n-torka na izlazu za svaku grupu	Jedna n-torka na izlazu za svaku u prozor ulaznu n-torku

```

SELECT  sifPred
        , sifStud
        , datumRok
        , COUNT(*) OVER (PARTITION BY
                           sifPred, datumRok) brIsp
        , AVG(ocjena) OVER (PARTITION BY
                             sifPred) prOcj
FROM  ispit
  
```

sif Pred	sif Stud	datumRok	ocjena	br Isp	pr Ocj
1	1	01.06.2020	1	2	4.00
1	3	01.06.2020	5	2	4.00
1	1	01.07.2020	4	4	4.00
1	2	01.07.2020	5	4	4.00
1	4	01.07.2020	4	4	4.00
1	5	01.07.2020	5	4	4.00
2	1	01.07.2020	3	2	2.50
2	2	01.07.2020	2	2	2.50

Prozor (Window)

Tranzijentni skup n-torki pomoću kojeg se definiraju:

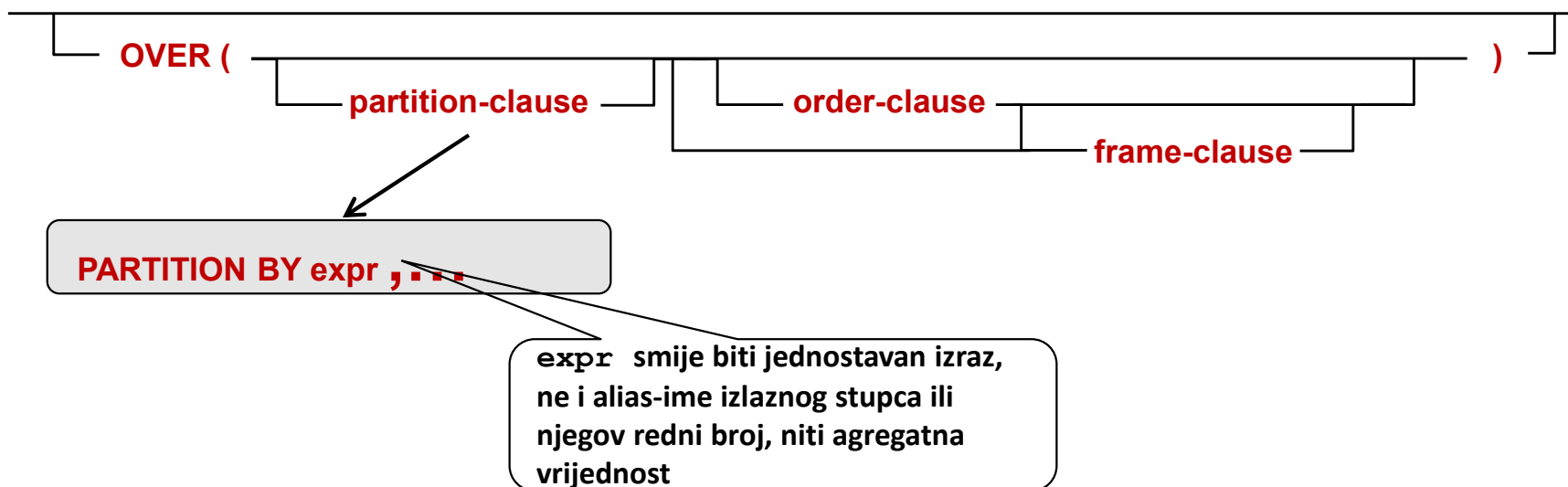
- Particija (partition)
 - Fiksna (ne može biti pomična)
 - Može sadržavati okvir(e)
- Okvir (frame)
 - Može se pomicati unutar particije
 - Ne može zahvatiti više particija
- **U istoj SELECT naredbi moguće je različite izraze izračunavati temeljem n-torki sadržanih u različito definiranim particijama/prozorima**

PostgreSQL: Particija

- Definira se pomoću *partition-clause*, odnosno, PARTITION BY u OVER ()
- Omogućuje podjelu n-torki relacije, slično kao GROUP BY
- Svaka n-torka pripada jednoj particiji
- Bez PARTITION BY cijela relacija je jedna particija

```
SELECT sifPred
      , sifStud
      , ocjena
      , AVG(ocjena) OVER (PARTITION BY sifPred, datumRok)
FROM ispit
```

Function
(args)



PostgreSQL: Particija – primjeri 1

ispit

sifPred	sifStud	datumRok	ocjena
1	1	01.06.2020	1
1	3	01.06.2020	5
1	1	01.07.2020	4
1	2	01.07.2020	5
1	4	01.07.2020	4
1	5	01.07.2020	5
2	1	01.07.2020	3
2	2	01.07.2020	2

```
SELECT AVG(ocjena) OVER ()  
FROM ispit;
```

Cijela relacija je jedna particija.

avg
3.625
3.625
3.625
3.625
3.625
3.625
3.625
3.625

PostgreSQL: Particija – primjeri 2

```
SELECT sifPred, sifStud, datumRok, ocjena
      , AVG(ocjena) OVER(PARTITION BY sifPred)
FROM ispit
ORDER BY sifPred, datumRok, sifStud;
```

P1

Particiji pripadaju n-torke s jednakom sifPred.

sifPred	sifStud	datumRok	ocjena	avg
1	1	01.06.2020	1	4.000
1	3	01.06.2020	5	4.000
1	1	01.07.2020	4	4.000
1	2	01.07.2020	5	4.000
1	4	01.07.2020	4	4.000
1	5	01.07.2020	5	4.000
2	1	01.07.2020	3	2.500
2	2	01.07.2020	2	2.500

P2

```
SELECT sifPred, sifStud, datumRok, ocjena
      , AVG(ocjena) OVER (PARTITION BY sifPred
                           , datumRok)
FROM ispit
ORDER BY sifPred, datumRok, sifStud;
```

P1

P2

P3

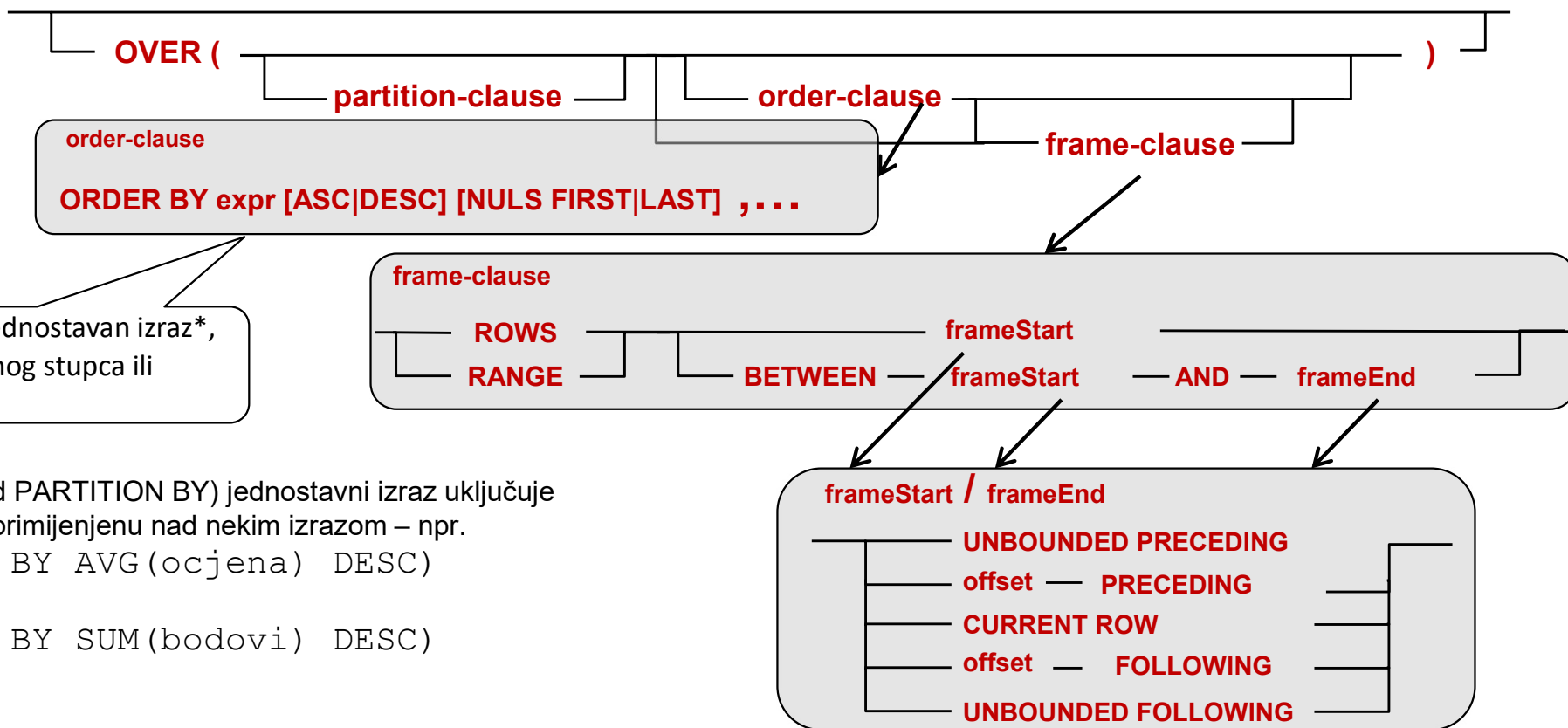
Particiji pripadaju n-torke s jednakom sifPred i datumRok.

sifPred	sifStud	datumRok	ocjena	avg
1	1	01.06.2020	1	3.000
1	3	01.06.2020	5	3.000
1	1	01.07.2020	4	4.500
1	2	01.07.2020	5	4.500
1	4	01.07.2020	4	4.500
1	5	01.07.2020	5	4.500
2	1	01.07.2020	3	2.500
2	2	01.07.2020	2	2.500

PostgreSQL: Okvir (frame)

- Definira se uvijek u odnosu na tekuću n-torku
- Okvir se može pomicati kako se pomičemo po djelatnom skupu
- Svaka n-torka pripada okviru, okvir pripada particiji
- Definira se pomoću *order-clause* i *frame-clause* u OVER ()
- Bez frame i ORDER BY dijela, okvir svake n-torke je cijela particija

Function
(args)



*ovdje (za razliku od PARTITION BY) jednostavni izraz uključuje agregatnu funkciju primijenjenu nad nekim izrazom – npr.
`OVER (ORDER BY AVG(ocjena) DESC)`
ili
`OVER (ORDER BY SUM(bodovi) DESC)`

PostgreSQL: Okvir definiran samo pomoću *order-clause*

- ako OVER sadrži *order-clause* (ORDER BY) ali NE i *frame-clause*, okviru trenutne n-torke pripadaju sve n-torke od početka particije do trenutne uključujući sve njene vršnjake (eng. peers)
- vršnjačke n-torke imaju jednake vrijednosti izraza iz ORDER BY dijela definicije okvira (u nekim slučajevima smije biti samo 1 izraz u ORDER BY)
- vrijednost agregatne funkcije za n-torku se računa temeljem svih n-torki koji pripadaju njenom okviru

```
SELECT sifPred, sifStud, datumRok
      , ocjena
      , AVG(ocjena) OVER (ORDER BY sifPred)
FROM ispit
ORDER BY sifPred, datumRok, sifStud;
```

- Particija je cijela relacija
- Vršnjačke n-torke imaju jednaku vrijednost atributa sifPred
- Okviru trenutne n-torke pripadaju n-torke od početka particije do trenutne uključujući sve njene vršnjake

P1

sifPred	sifStud	datumRok	ocjena	avg
1	1	01.06.2020	1	4.000
1	3	01.06.2020	5	4.000
1	1	01.07.2020	4	4.000
1	2	01.07.2020	5	4.000
1	4	01.07.2020	4	4.000
1	5	01.07.2020	5	4.000
2	1	01.07.2020	3	3.625
2	2	01.07.2020	2	3.625

Vršnjaci

Vršnjaci

PostgreSQL: Okvir definiran samo pomoću *order-clause* - primjer

```
SELECT *, AVG(ocjena)
      OVER (PARTITION BY sifPred
            ORDER BY datumRok)
FROM ispit
ORDER BY sifPred, datumRok;
```

```
SELECT *, AVG(ocjena)
      OVER (PARTITION BY sifPred
            ORDER BY ocjena)
FROM ispit
ORDER BY sifPred, ocjena;
```

Particiji pripadaju n-torke s jednakom sifPred.

Vršnjačke n-torke imaju jednaku vrijednost atributa **datumRok**

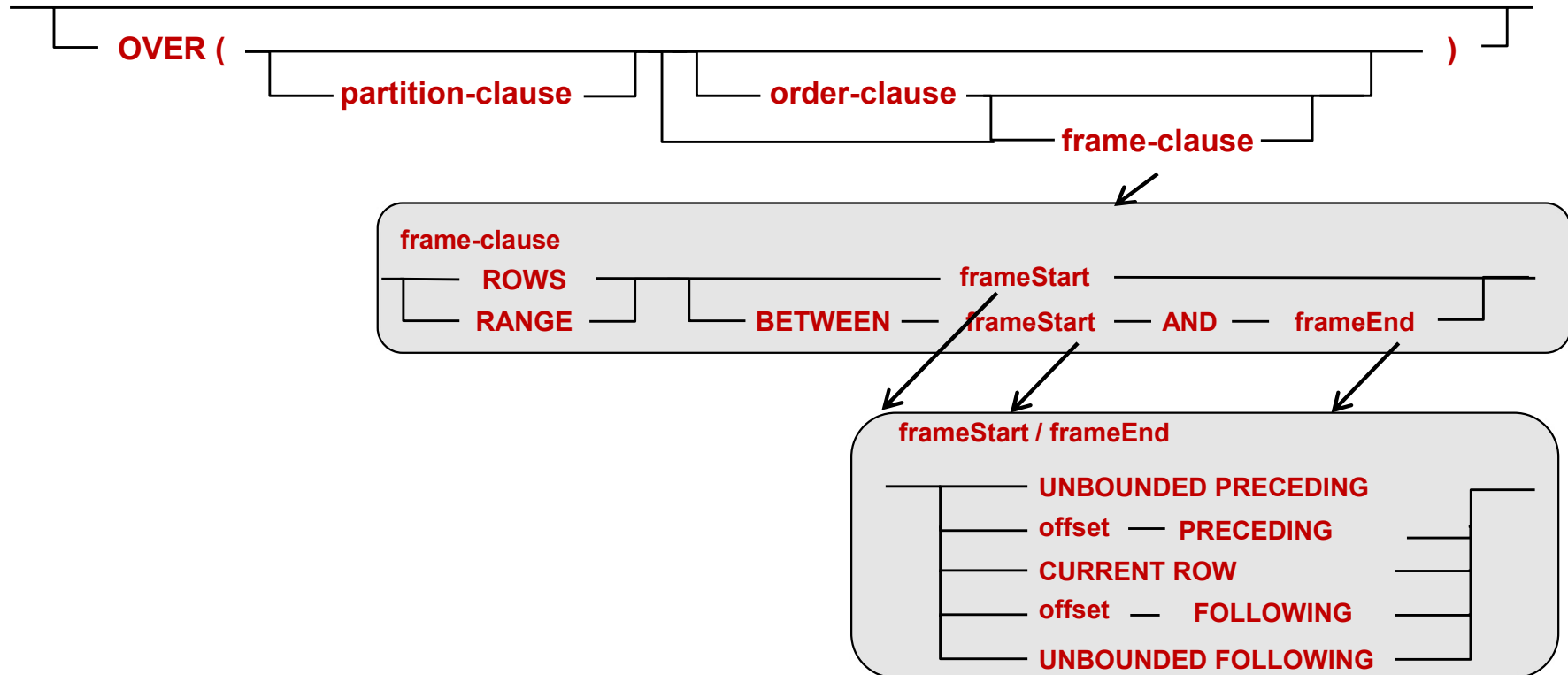
Vršnjačke n-torke imaju jednaku vrijednost atributa **ocjena**

Okviru trenutne n-torke pripadaju sve n-torke od početka particije do trenutne n-torke uključujući sve njene vršnjake

	sifPred	sifStud	datumRok	ocjena	avg
P1	1	1	01.06.2020	1	3.000
	1	3	01.06.2020	5	3.000
	1	1	01.07.2020	4	4.000
	1	4	01.07.2020	4	4.000
	1	2	01.07.2020	5	4.000
	1	5	01.07.2020	5	4.000
P2	2	2	01.07.2020	2	2.500
	2	1	01.07.2020	3	2.500

	sifPred	sifStud	datumRok	ocjena	avg
P1	1	1	01.06.2020	1	1.000
	1	1	01.07.2020	4	3.000
	1	4	01.07.2020	4	3.000
	1	3	01.06.2020	5	4.000
	1	2	01.07.2020	5	4.000
	1	5	01.07.2020	5	4.000
P2	2	5	01.07.2020	2	2.000
	2	1	01.07.2020	3	2.500

PostgreSQL: Okvir - *frame_clause*



PostgreSQL: Okvir - *frame_clause*

frame-clause

ROWS **frameStart**
RANGE **BETWEEN** **frameStart** **AND** **frameEnd**

	ROWS	RANGE
vršnjačke ntorkke	ne postoje	postoje
poredak (ORDER BY)	ne mora biti naveden	mora biti naveden ako se koristi frame-clause s offset mora sadržavati točno jedan izraz inače može sadržavati proizvoljno mnogo izraza
ntorka pripada okviru tekuće ntorkke ako	je u specificiranom rasponu – navodi se broj n-torki koje prethode ili slijede tekuću	vrijednost (jednog) izraza navedenog u ORDER BY za ntorku je u specificiranom rasponu vrijednosti

PostgreSQL: Okvir - *frame_clause*

- Okviru trenutne ntorke pripadaju ntorke od ***frameStart*** do ***frameEnd***
 - ***FrameStart*** i ***FrameEnd*** mogu biti :
 - UNBOUNDED PRECEDING
 - offset PRECEDING
 - CURRENT ROW
 - offset FOLLOWING
 - UNBOUNDED FOLLOWING
 - Ako se izostavi ***frameEnd*** onda je podrazumijevana vrijednost za ***frameEnd*** - CURRENT ROW
- Odnosno, umjesto:
- BETWEEN ***frameStart*** AND CURRENT ROW
- može se koristiti samo ***frameStart***

PostgreSQL od verzije 11 ima 3 vrste okvira: { RANGE | ROWS | GROUPS } .

[link](#) – za one koji žele znati više

PostgreSQL: Okvir - *frame_clause*

<i>frameStart</i>	ROWS RANGE
UNBOUNDED PRECEDING	Okvir trenutne ntorkke počinje prvom ntorkom particije.

<i>frameEnd</i>	ROWS RANGE
UNBOUNDED FOLLOWING	Okvir trenutne ntorkke završava zadnjom ntorkom particije.

<i>frameStart/ frameEnd</i>	ROWS	RANGE
CURRENT ROW	frameStart odnosno frameEnd je trenutna n-torka	okvir počinje prvom odnosno završava zadnjom vršnjačkom ntorkom trenutne ntorkke
offset PRECEDING offset FOLLOWING	Offset je izraz koji ne sadrži variable, agregatne funkcije niti „window funkcije” Tip podatka izraza je ne negativan cijeli broj. Predstavlja broj ntorki koje prethode odnosno slijede trenutnu.	Tip podatka je određen tipom (jedinog) izraza u ORDER BY. Predstavlja najveću razliku između vrijednosti tog izraza za trenutnu ntorku i vrijednost istog izraza za prethodne odnosno sljedeće ntorkke.

PostgreSQL: Okvir

Podrazumijevana (default) vrsta okvira je:

RANGE UNBOUNDED PRECEDING

odnosno:

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

Ako se

- koristi samo *order_clause* (i default vrsta okvira), okviru trenutne ntorke pripadaju ntorke od početka particije do zadnje vršnjačke ntorke.
- ne koristi ni *order_clause* okviru trenutne ntorke pripadaju sve ntorke iz particije (u tom slučaju sve ntorke particije postaju vršnjačke).

PostgreSQL: Okvir definiran pomoću ROWS - primjeri

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	sve n-torke od početka particije do tekuće n-torke
ROWS BETWEEN 1 PRECEDING AND 1 PRECEDING	prethodna n-torka
ROWS BETWEEN 1 FOLLOWING AND 1 FOLLOWING	sljedeća n-torka
ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING	Prethodna, trenutna i sljedeća n- torka

PostgreSQL: Particije i okviri

```
SELECT *, AVG(ocjena)
  OVER (PARTITION BY sifPred
        ORDER BY datumRok
        RANGE BETWEEN UNBOUNDED PRECEDING
              AND CURRENT ROW) pros
FROM ispit
ORDER BY sifPred, datumRok, pros;
```

```
SELECT *, AVG(ocjena)
  OVER (PARTITION BY sifPred
        ORDER BY datumRok
        ROWS BETWEEN UNBOUNDED PRECEDING
              AND CURRENT ROW) pros
FROM ispit
ORDER BY sifPred, datumRok, pros;
```

RANGE – postoje **vršnjačke n-torke**.

Vršnjačke n-torke imaju jednaku vrijednost atributa **datumRok**.

Okviru trenutne n-torke pripadaju sve n-torke počev od početka particije do trenutne n-torke **uključujući sve njene vršnjake**

ROWS – NE postoje vršnjačke n-torke.

Okviru trenutne n-torke pripadaju sve n-torke od početka particije do trenutne n-torke

	sifPred	sifStud	datumRok	ocjena	avg
P1	1	1	01.06.2020	1	3.000
	1	3	01.06.2020	5	3.000
	1	1	01.07.2020	4	4.000
	1	4	01.07.2020	5	4.000
	1	2	01.07.2020	4	4.000
	1	5	01.07.2020	5	4.000
P2	2	2	01.07.2020	2	2.500
	2	1	01.07.2020	3	2.500

	sifPred	sifStud	datumRok	ocjena	avg
P1	1	1	01.06.2020	1	1.000
	1	3	01.06.2020	5	3.000
	1	1	01.07.2020	4	3.333
	1	4	01.07.2020	5	3.750
	1	2	01.07.2020	4	3.800
	1	5	01.07.2020	5	4.000
P2	2	2	01.07.2020	2	2.500
	2	1	01.07.2020	3	3.000

Zašto nije
2.000 i 2.500
umjesto
2.500 i 3.000?

PostgreSQL: Definiranje okvira pomoću RANGE - primjer 1

dohodakDjel

	sifDjelatnik	datumDohodak	iznos		sifDjelatnik	datumDohodak	iznos	iznosCum
P1	1	01.12.2024	3000.00	→	1	01.12.2024	3000.00	7000.00
	1	01.01.2025	4000.00		1	01.01.2025	4000.00	7000.00
	1	01.06.2025	5000.00		1	01.06.2025	5000.00	6000.00
	1	15.06.2025	1000.00		1	15.06.2025	1000.00	6000.00
P2	1	01.08.2025	5000.00		1	01.08.2025	5000.00	11000.00
	1	01.09.2025	6000.00		1	01.09.2025	6000.00	11000.00
	2	01.07.2025	7000.00		2	01.07.2025	7000.00	7000.00
	2	01.09.2025	5000.00		2	01.09.2025	5000.00	5000.00

```
SELECT sifDjelatnik, datumDohodak,  
       iznos,  
       SUM(iznos) OVER (PARTITION BY sifDjelatnik  
                        ORDER BY datumDohodak  
                        RANGE BETWEEN '1 MON'::INTERVAL PRECEDING AND  
                                   '1 MON'::INTERVAL FOLLOWING) iznos3M  
FROM dohodakDjel  
ORDER BY sifDjelatnik, EXTRACT (MONTH FROM datumDohodak)
```

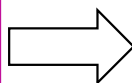
Dodavanjem i oduzimanjem jednog mjeseca datumu tekuće n-torke stvara se okvir od 2 mjeseca (npr. 01.11.2024. – 01.01.2025.).

PostgreSQL: Definiranje okvira pomoću RANGE - primjer 2

boravak

P1

oznDvorana	sifOsoba	datumVrijemePoc
A101	1	01.10.2025 08:04:36
A101	2	01.10.2025 09:14:00
A101	3	01.10.2025 11:05:36
A101	4	01.10.2025 11:14:01
M5	1	01.10.2025 08:14:50
M5	2	01.10.2025 12:14:45



oznDvorana	datum	vrijeme	cnt5H
A101	01.10.2025	08:04:36	2
A101	01.10.2025	09:14:00	3
A101	01.10.2025	11:05:36	3
A101	01.10.2025	11:14:01	2
M5	01.10.2025	08:14:50	1
M5	01.10.2025	12:14:45	1

P2

```
SELECT oznDvorana,  
       datumVrijemePoc::DATE datum,  
       datumVrijemePoc::TIME vrijeme,  
       COUNT(*) OVER(PARTITION BY oznDvorana  
                     ORDER BY datumVrijemePoc  
                     RANGE BETWEEN '2 h'::INTERVAL PRECEDING AND  
                                   '2 h'::INTERVAL FOLLOWING)  
FROM boravak  
ORDER BY oznDvorana, datum, vrijeme
```

ORDER BY izraz je TIMESTAMP tipa.

Okvir trenutne ntorkke obuhvaća vremenske trenutke u rasponu od 4 sata: [2 sata prije, 2 sata poslije]. TIMESTAMP trenutne ntorkke pripada tom intervalu.

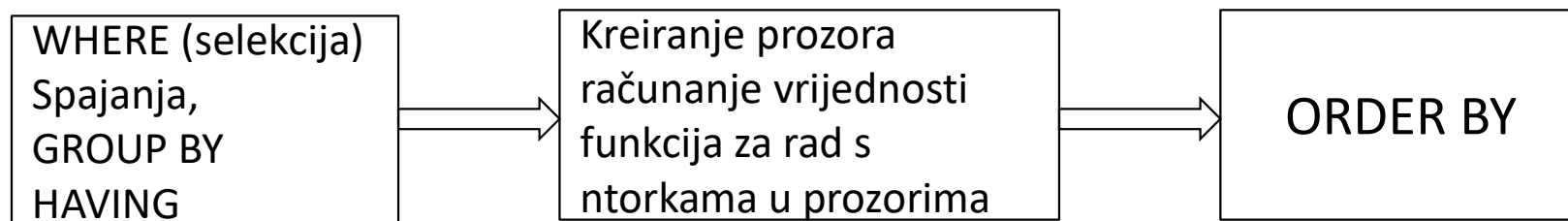
Redoslijed aktivnosti pri obavljanju upita

Prozore je moguće koristiti isključivo u

- SELECT i ORDER BY

ne u npr.

- WHERE, GROUP BY i HAVING



Funkcije za rad s prozorima mogu koristiti izračunate vrijednosti agregatnih funkcija (COUNT, AVG, SUM,...) nakon procesiranja FROM, WHERE, GROUP BY i HAVING dijelova SELECT naredbe

Posljedica (i to odlična): mogućnost obavljanja ugniježđenih agregacija:

- **AVG (SUM (bod))**
- **MIN (SUM (bod))**
- ...

PostgreSQL: Ugniježdene agregacije – primjer 1

Za predmet i ispitni rok odrediti kumulativan broj ispita koji uključuje sve ispite s tog i neposredno prethodnog ispitnog roka iz istog predmeta.

ispit

P1

sifPred	sifStud	datumRok	ocjena
1	1	01.06.2020	1
1	3	01.06.2020	5
1	1	01.07.2020	4
1	2	01.07.2020	5
1	4	01.07.2020	4
1	5	01.07.2020	5
1	6	01.09.2020	3
1	7	01.09.2020	4
1	8	01.09.2020	5
2	1	01.07.2020	3
2	2	01.07.2020	2

P2

sifPred	datumRok	kumBrispit
1	01.06.2020	2
1	01.07.2020	6
1	01.09.2020	7
2	01.07.2020	2

Mogu li s
RANGE?

```
SELECT sifPred, datumRok
, SUM(COUNT(*)) OVER (PARTITION BY sifPred ORDER BY datumRok
ROWS 1 PRECEDING) kumBrIspit

FROM ispit
GROUP BY sifPred, datumRok
ORDER BY sifPred, datumRok
```

Što bi
računao bez
frame clause?



Ugrađene funkcije za rad s prozorima (Window functions)

Prema SQL standardu funkcija za rad s prozorima može biti:

1. funkcija za rangiranje
2. funkcija za distribuciju
3. funkcija za određivanje n-torke temeljem pozicije (row number)
4. agregatna funkcija na razini prozora

PostgreSQL: Ugrađene funkcije za rad s prozorima

- Funkcije za rangiranje:
 - `row_number()`
 - `rank()`
 - `dense_rank()`
 - `percent_rank()`
- Funkcije za distribuciju:
 - `cume_dist()`
 - `ntile()`
- Funkcija za određivanje n-torke temeljem pozicije
 - `lag()`
 - `lead()`
 - `first_value()`
 - `last_value()`
 - `nth_value()`

PostgreSQL: Funkcije za rangiranje

<code>row_number ()</code>	Redni broj n-torke u particiji, počinje od 1 za svaku particiju
<code>rank ()</code>	Rang n-torke u particiji, za jednake vrijednosti vraća jednak rang
<code>dense_rank ()</code>	Jednako kao <i>rank ()</i> ali bez "rupa" u slučaju jednako rangiranih n-torki
<code>percent_rank()</code>	relativni rang n-torke: $(rang-1)/(ukupno\ n-torki - 1)$

PostgreSQL: Funkcije za rangiranje - primjeri

ispit

sifPred	sifStud	datumRok	ocjena
1	1	01.06.2020	1
1	3	01.06.2020	5
1	1	01.07.2020	4
1	2	01.07.2020	5
1	4	01.07.2020	3
1	5	01.07.2020	2
2	1	01.07.2020	3
2	2	01.07.2020	2

```
SELECT sifPred, sifStud, datumRok, ocjena
, row_number() OVER (ORDER BY ocjena DESC) as rowNum
, rank() OVER (ORDER BY ocjena DESC) as rang
, dense_rank() OVER (ORDER BY ocjena DESC) as dRang
FROM ispit
ORDER BY ocjena DESC
```

sifPred	sifStud	datumRok	ocjena	rowNum	rang	dRang
1	3	01.06.2020	5	1	1	1
1	2	01.07.2020	5	2	1	1
1	1	01.07.2020	4	3	3	2
2	1	01.07.2020	3	4	4	3
1	4	01.07.2020	3	5	4	3
2	2	01.07.2020	2	6	6	4
1	5	01.07.2020	2	7	6	4
1	1	01.06.2020	1	8	8	5

```
SELECT sifPred, sifStud, datumRok, ocjena
, row_number() OVER (PARTITION BY datumRok
ORDER BY ocjena DESC) as rowNum
, rank() OVER (PARTITION BY datumRok
ORDER BY ocjena DESC) as rang
, dense_rank() OVER (PARTITION BY datumRok
ORDER BY ocjena DESC) as dRang
FROM ispit
ORDER BY datumRok, ocjena DESC
```

sifPred	sifStud	datumRok	ocjena	rowNum	rang	dRang
1	3	01.06.2020	5	1	1	1
1	1	01.06.2020	1	2	2	2
1	2	01.07.2020	5	1	1	1
1	1	01.07.2020	4	2	2	2
1	4	01.07.2020	3	3	3	3
2	1	01.07.2020	3	4	3	3
2	2	01.07.2020	2	5	5	4
1	5	01.07.2020	2	6	5	4

PostgreSQL: Rangiranje temeljem agregacije - primjer

Obzirom na ukupno osvojene bodove studenta na predmetu odrediti

- rang studenta na predmetu i
- rang studenta ukupno, bez obzira na predmet.

Zapise poredati prema šifri predmeta i rangi unutar predmeta.

studProvjera

sifPred	sifStud	kratProvjera	bod
1	1	1DZ	5.00
1	2	1DZ	10.00
1	3	1DZ	5.00
1	1	MI	10.00
1	2	MI	5.00
1	3	MI	20.00
2	1	MI	20.00
2	2	MI	25.00
2	3	MI	30.00

```
SELECT sifPred, sifStud, SUM(Bod) ukBod
      , rank() OVER (PARTITION BY sifPred
                     ORDER BY SUM(bod) DESC) rangPred
      , rank() OVER (ORDER BY SUM(bod) DESC) rangGlob
FROM studProvjera
GROUP BY sifPred, sifStud
ORDER BY sifPred, SUM(bod) DESC
--ORDER BY sifPred, rank() OVER (PARTITION BY sifPred
--                                ORDER BY SUM(bod) DESC)
--ORDER BY sifPred, rangPred
```

sifPred	sifStud	ukBod	rangPred	rangGlob
1	3	25.00	1	2
1	1	15.00	2	5
1	2	15.00	2	5
2	3	30.00	1	1
2	2	25.00	2	2
2	1	20.00	3	4

PostgreSQL: Funkcije za određivanje n-torke temeljem pozicije

<code>lag(value any [,offset integer [,default any]])</code>	• Vraća vrijednost atributa (predanog kao argument) n-torke koja se nalazi u istoj particiji <i>offset</i> n-torki prije tekuće • Ako takva n-torka ne postoji, vraća <i>default</i> vrijednost • I <i>offset</i> i <i>default</i> se određuju u odnosu na tekuću n-torku. • Predodređene vrijednosti su 1 za <i>offset</i> i <i>NULL</i> za <i>default</i>
<code>lead(value any [,offset integer [,default any]])</code>	Jednako kao lag osim što n-torku traži <i>offset</i> n-torki nakon tekuće
<code>first_value(value any)</code>	Vraća vrijednost atributa (predanog kao argument) n-torke koja je prva u okviru
<code>last_value(value any)</code>	Vraća vrijednost atributa (predanog kao argument) n-torke koja je zadnja u okviru
<code>nth_value(value any, nth integer)</code>	• Vraća vrijednost atributa (predanog kao argument) koja je izračunata za n-torku na <i>n-toj</i> poziciji u okviru (počinje od 1) • Ako takva n-torka ne postoji, vraća <i>NULL</i> vrijednost

PostgreSQL: Određivanje n-torke temeljem pozicije - primjer

Na temelju dnevnika pogrešaka donosi se odluka o prioritetu popravljjanja pogrešaka.

Da bi se pogreška našla na listi prioriteta, mora se istom korisniku pojaviti barem 3 puta u periodu od 15 minuta. Odrediti treba li pogrešku 75 staviti na listu prioriteta.

dnevnikGreska

sifDnevnik	sifGreska	vrijemeGreska	korisnik
10566	75	25.07.2020 10:18:00	pjuric
10567	75	25.07.2020 10:18:20	tkralj
10568	75	25.07.2020 10:18:50	tkralj
10569	75	25.07.2020 10:23:00	tkralj
10570	75	25.07.2020 10:28:30	tkralj
10571	75	25.07.2020 10:31:00	tkralj
10572	75	25.07.2020 10:33:40	tkralj

Uzeti u obzir samo greške 75

WHERE sifGreska = 75

Gledati zajedno greške istog korisnika

PARTITION BY korisnik

Svakom zapisu pronaći zapis koji pripada istom korisniku (u istoj je particiji), a obzirom na vrijeme greške, je smješten 2 pozicije prije (zajedno s promatranim zapisom radi se o točno 3 greške).

LAG(vrijemeGreska, 2)

```
SELECT sifDnevnik, sifGreska, korisnik, vrijemeGreska
      , LAG(vrijemeGreska, 2) OVER (PARTITION BY korisnik
                                   ORDER BY vrijemeGreska ASC) AS istaGreska2pozPrije
FROM dnevnikGreska
WHERE sifGreska = 75;
```

sifDnevnik	sifGreska	korisnik	vrijemeGreska	istaGreska2pozPrije
10566	75	pjuric	25.07.2020 10:18:00	
10567	75	tkralj	25.07.2020 10:18:20	
10568	75	tkralj	25.07.2020 10:18:50	
10569	75	tkralj	25.07.2020 10:23:00	25.07.2020 10:18:20
10570	75	tkralj	25.07.2020 10:28:30	25.07.2020 10:18:50
10571	75	tkralj	25.07.2020 10:31:00	25.07.2020 10:23:00
10572	75	tkralj	25.07.2020 10:33:40	25.07.2020 10:28:30

PostgreSQL: Određivanje n-torke temeljem pozicije - primjer

sifDnevnik	sifGreska	korisnik	vrijemeGreska	istaGreska2pozPrije
10566	75	pjuric	25.07.2020 10:18:00	
10567	75	tkralj	25.07.2020 10:18:20	
10568	75	tkralj	25.07.2020 10:18:50	
10569	75	tkralj	25.07.2020 10:23:00	25.07.2020 10:18:20
10570	75	tkralj	25.07.2020 10:28:30	25.07.2020 10:18:50
10571	75	tkralj	25.07.2020 10:31:00	25.07.2020 10:23:00
10572	75	tkralj	25.07.2020 10:33:40	25.07.2020 10:28:30

Postoje li zapisi za koje vrijedi da se u manje od 15 minuta ista greška istom korisniku pojavila još 2 puta?

```
SELECT *
FROM
  (SELECT sifDnevnik, sifGreska, korisnik, vrijemeGreska
    , LAG(vrijemeGreska, 2) OVER
      (PARTITION BY korisnik
        ORDER BY vrijemeGreska ASC) AS istaGreska2PozPrije
    FROM dnevnikGreska
    WHERE sifGreska = 75) AS greska75
WHERE (greska75.vrijemeGreska -
  greska75.istaGreska2PozPrije) < '15 minutes'::INTERVAL;
```

sifDnevnik	sifGreska	korisnik	vrijemeGreska	istaGreska2pozPrije
10570	75	tkralj	25.07.2020 10:28:30	25.07.2020 10:18:00
10571	75	tkralj	25.07.2020 10:31:00	25.07.2020 10:18:50
10572	75	tkralj	25.07.2020 10:33:40	25.07.2020 10:23:00



Napredni SQL - teme

1. Pivotiranje
2. Funkcije za rad s podacima u prozoru
3. **Rekurzivni upiti i CTE (Common Table Expression)**

Motivacijski primjer 1

Pronađi sve osobe s kojima je **ana123@hotmail.com** povezana direktno ili preko prijatelja. Uzeti u obzir samo prijateljstva u kojima je **ana123@hotmail.com** (odnosno njeni prijatelji i td.) inicijator (osoba1).

foaf

Friend of a friend

osoba1	osoba2
...	...
ana123@hotmail.com	zecG@gmail.com
zecG@gmail.com	iva.malic@fer.hr
klaraB@gmail.com	ana123@hotmail.com
zecG@gmail.com	jezV@hotmail.com
Iva.malic@fer.hr	jezV@hotmail.com
...	...

Prijatelji

Prijatelji
prijatelja

Prijatelji
prijatelja
prijatelja

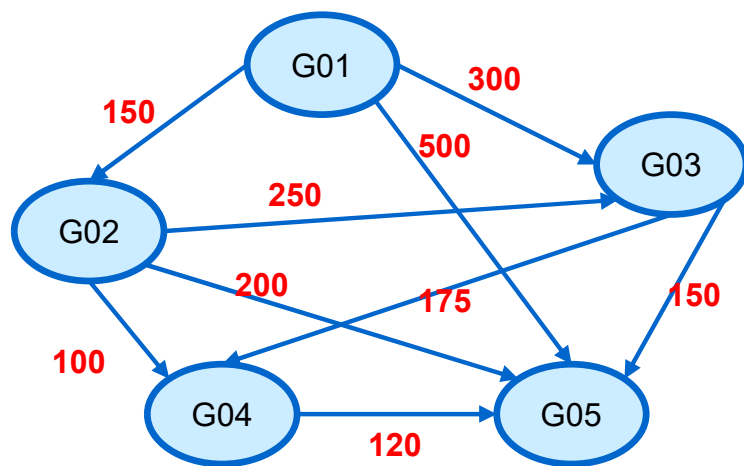
```
SELECT osoba1, osoba2
FROM foaf foaf1
WHERE foaf1.osoba1 = 'ana123@hotmail.com'
UNION
SELECT foaf1.osoba1, foaf2.osoba2
FROM foaf foaf1, foaf foaf2
WHERE foaf1.osoba2 = foaf2.osoba1
AND foaf1.osoba1 = 'ana123@hotmail.com'
UNION
SELECT foaf1.osoba1, foaf3.osoba2
FROM foaf foaf1, foaf foaf2, foaf foaf3
WHERE foaf1.osoba2 = foaf2.osoba1
AND foaf2.osoba2 = foaf3.osoba1
AND foaf1.osoba1 = 'ana123@hotmail.com'
UNION ...
```

SQL upitom bismo znali pronaći sve prijatelje osobe **ana123@hotmail.com**, sve prijatelje prijatelja itd.

- Znali bismo napisati (nespretno, nepregledano) upit za unaprijed zadanu "dubinu" veze.
- Ne znamo riješiti problem za nepoznatu/promjenjivu "dubinu" veze.

Motivacijski primjer 2

Pronađi najjeftiniji let između polazišta A i odredišta B.



Ako je A = G01, a B = G05 mogućih ruta i cijena je više:

ruta	cijena
G01 – G05	500
G01 – G02 – G05	150+200 = 350
G01 – G02 – G03 – G05	150+250+150 = 550
...	...

Uzevši u obzir moguća presjedanja i činjenicu da manje presjedanja ne garantira nižu cijenu problem se svodi na rekurzivnu pretragu relacije *let*.

let

polaziste	odrediste	cijena
G01	G02	150
G01	G03	300
G01	G05	500
G02	G03	250
G02	G04	100
G02	G05	200
G03	G04	175
G03	G05	150
G04	G05	120

Common Table Expressions (CTE)

- SQL:1999 (SQL3) standard definira WITH naredbu:

```
WITH R1 AS (query1),  
     R2 AS (query2),  
     ...,  
     Rn AS (queryn),  
<upit koji uključuje R1, R2,...,Rn (i druge relacije)>
```

Ideja:

1. Odrediti sadržaj relacija R1, R2,..., Rn i pohraniti ga u privremene relacije (TEMP TABLE)
2. Procesirati upit koji uključuje R1, R2,..., Rn i druge relacije
3. Ukloniti R1, R2,..., Rn

Ovakvi izrazi se nazivaju Common Table Expressions (CTE)

PostgreSQL: evaluacija rekurzivnog upita

```
WITH RECURSIVE
  R AS (upit
        UNION
        upit koji obavezno uključuje R)
<upit koji uključuje R (i druge relacije)>
```

→ nerekurzivni
UNION
→ rekurzivni

1. Evaluiraj nerekurzivni upit (**upit**).
Zapise prethodnog koraka uključi u rezultat rekurzivnog upita **R** i, u privremenu (temporary) **radnu** tablicu.
2. Dok postoje zapisi u *radnoj* tablici ponavljaj:
 - a) Evaluiraj rekurzivni upit koristeći sadržaj **radne** tablice za rekurzivno spajanje (umjesto **R**)
Zapise prethodnog koraka uključi u rezultat rekurzivnog upita **R** i, u privremenu (temp) tablicu **međurezultata**.
 - b) Isprazni **radnu** tablicu i napuni je sadržajem tablice **međurezultata**.
Isprazni tablicu **međurezultata**.

Primjer 1 - foaf

Pronađi sve osobe s kojima je osoba **ana123@hotmail.com** povezana direktno ili preko prijatelja. Uzeti u obzir samo prijateljstva u kojima je **ana123@hotmail.com** (odnosno njeni prijatelji i td.) inicijator (osoba1).

foaf

osoba1	osoba2
...	...
ana123@hotmail.com	zecG@gmail.com
zecG@gmail.com	iva.malic@fer.hr
klaraB@gmail.com	ana123@hotmail.com
zecG@gmail.com	jezV@hotmail.com
Iva.malic@fer.hr	jezV@hotmail.com
...	...

Neposredni prijatelji:

osoba1	osoba2
ana123@hotmail.com	zecG@gmail.com

Prijatelji prijatelja, prijatelja,... :

osoba1	osoba2	
ana123@hotmail.com	iva.malic@fer.hr	preko zecG@gmail.com
ana123@hotmail.com	jezV@hotmail.com	preko zecG@gmail.com
ana123@hotmail.com	jezV@hotmail.com	preko zecG@gmail.com $\Rightarrow$ iva.malic@fer.hr

PostgreSQL: Primjer 1 - foaf

Result (same as the content of the foaf):

```
WITH RECURSIVE prijatelji(prijatelj1,  
                           prijatelj2) AS  
(  
    SELECT osoba1, osoba2,  
           FROM foaf  
)  
SELECT * FROM prijatelji
```

Using UNION

```
WITH RECURSIVE prijatelji (prijatelj1, prijatelj2) AS  
(  
    SELECT osoba1  
          , osoba2  
    FROM foaf  
    UNION  
    SELECT prijatelji.prijatelj1  
          , foaf.osoba2,  
    FROM prijatelji, foaf  
    WHERE prijatelji.prijatelj2 = foaf.osoba1  
)  
SELECT * FROM prijatelji
```

Gdje je ntorka
(zecG@gmail.com, jezV@hotmail.com)?

UNION ALL?

prijatelj1	prijatelj2
ana123@hotmail.com	zecG@gmail.com
zecG@gmail.com	iva.malic@fer.hr
klaraB@gmail.com	ana123@hotmail.com
zecG@gmail.com	jezV@hotmail.com
Iva.malic@fer.hr	jezV@hotmail.com

1. iteracija

2. iteracija

3. iteracija

friend1	friend2
ana123@hotmail.com	zecG@gmail.com
zecG@gmail.com	iva.malic@fer.hr
klaraB@gmail.com	ana123@hotmail.com
zecG@gmail.com	jezV@hotmail.com
Iva.malic@fer.hr	jezV@hotmail.com
ana123@hotmail.com	iva.malic@fer.hr
ana123@hotmail.com	jezV@hotmail.com
klaraB@gmail.com	zecG@gmail.com
klaraB@gmail.com	iva.malic@fer.hr
klaraB@gmail.com	jezV@hotmail.com

PostgreSQL: Primjer 1 - foaf

```
WITH RECURSIVE prijatelji(prijatelj1, prijatelj2)
AS
(
    SELECT osoba1, osoba2
      FROM foaf

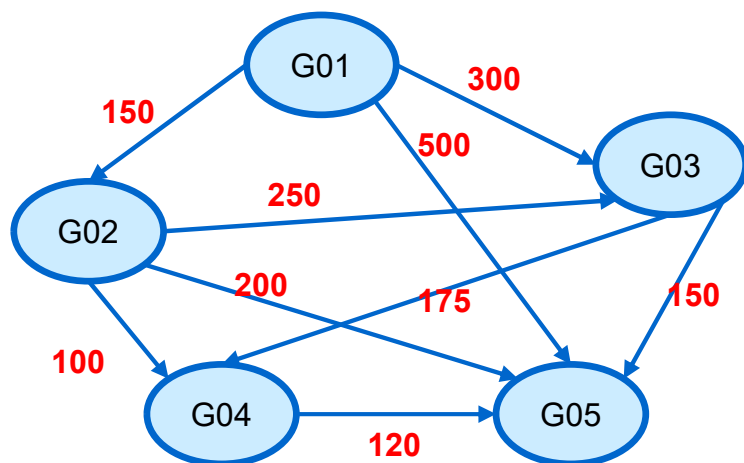
    UNION 

    SELECT prijatelji.prijatelj1
       , foaf.osoba2  as prijatelj2
      FROM prijatelji, foaf
     WHERE prijatelji.prijatelj2 = foaf.osoba1
)
SELECT * FROM prijatelji
WHERE prijatelj1 = 'ana123@hotmail.com'
```

prijatelj1	prijatelj2
ana123@hotmail.com	zecG@gmail.com
ana123@hotmail.com	iva.malic@fer.hr
ana123@hotmail.com	jezV@hotmail.com

Primjer 2 – najjeftiniji plan leta

Pronađi sve moguće planove letenja i pripadnu cijenu između polazišta G01 i odredišta G05. Prikaži i broj presjedanja.



Rezultat (mogući):

planLeta	ukCijena	brPresj
G01-->G05	500	0
G01-->G02-->G05	350	1
G01-->G03-->G05	450	1
G01-->G02-->G03-->G05	550	2
G01-->G02-->G04-->G05	370	2
G01-->G03-->G04-->G05	595	2
G01-->G02-->G03-->G04-->G05	695	3

let

polaziste	odrediste	cijena
G01	G02	150
G01	G03	300
G01	G05	500
G02	G03	250
G02	G04	100
G02	G05	200
G03	G04	175
G03	G05	150
G04	G05	120

Primjer 2 – najjeftiniji plan leta

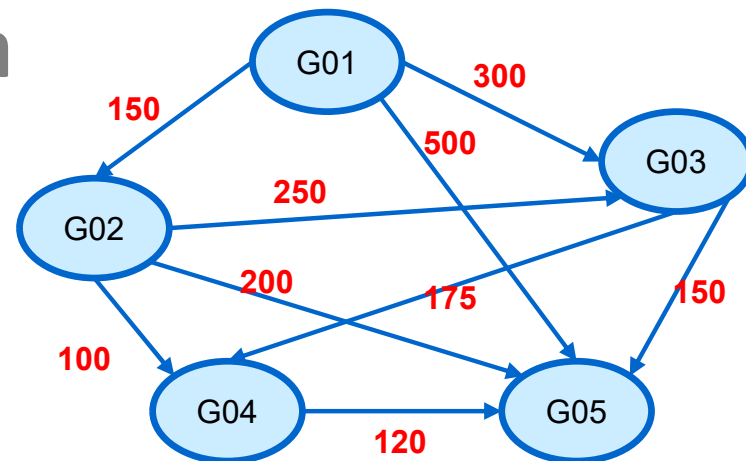
Ideja:

Pomoću rekurzivnog upita stvoriti privremenu relaciju:

```
planoviL (polaziste,
          odrediste,
          planLeta,
          ukCijena,
          brPresj)
```

gdje je

planLeta znakovni niz oblika G01->G02->G03->G04->G05
ukCijena ukupna cijela danog plana leta (150+250+175+120=695)
brPresj broj presjedanja (-1+1+1+1+1=3)



1. korak

U sviPlanoviLetenja insertirati n-torke sljedećeg oblika:

polaziste	odrediste	planLeta	ukCijena	brPresj
G01	G01	G01	0	-1
G02	G02	G02	0	-1
G03	G03	G03	0	-1
G04	G04	G04	0	-1

3. korak

U sviPlanoviLetenja insertirati n-torke sljedećeg oblika:

polaziste	odrediste	planLeta	ukCijena	brPresj
G01	G03	G01->G02->G03	400	1
G01	G04	G01->G02->G04	250	1
G01	G05	G01->G02->G05	350	1
...	...	...	...	1

2. korak

U sviPlanoviLetenja insertirati n-torke sljedećeg oblika:

polaziste	odrediste	planLeta	ukCijena	brPresj
G01	G02	G01->G02	150	0
G01	G03	G01->G03	300	0
G01	G05	G01->G05	500	0
G02	G03	G02->G03	250	0
G02	G04	G02->G04	100	0
G02	G05	G02->G05	200	0
G03	G04	G03->G04	175	0
G03	G05	G03->G05	150	0
G04	G05	G04->G05	120	0

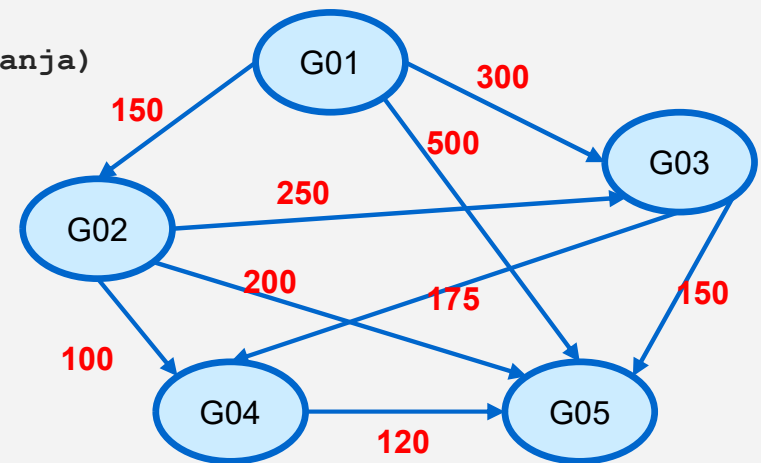
4. korak

U sviPlanoviLetenja insertirati n-torke ...

PostgreSQL: Primjer 2 –najjeftiniji plan leta

Pronađi sve moguće planove letenja i pripadnu cijenu između polazišta G01 i odredišta G05. Prikaži i broj presjedanja.

```
WITH RECURSIVE planoviL
(polaziste, odrediste, planLeta, ukCijena, brPresjedanja)
AS
( (SELECT DISTINCT
  polaziste,
  polaziste,
  CAST (polaziste AS VARCHAR(600)),
  0,
  -1
  FROM let
)
  UNION
  (SELECT planoviL.polaziste,
    let.odrediste,
    CAST (planoviL.planLeta || '->' || let.odrediste AS VARCHAR(600)),
    ukCijena + let.Cijena,
    brPresjedanja + 1
    FROM planoviL, let
    WHERE planoviL.odrediste = let.polaziste)
)
SELECT planLeta, ukCijena, brPresjedanja
  FROM planoviL
 WHERE polaziste = 'G01'
    AND odrediste = 'G05'
 ORDER BY ukCijena
```



let

polaziste	odrediste	cijena
G01	G02	150
G01	G03	300
G01	G05	500
...	...	...
G04	G05	120

Beskonačna rekurzija

foaf

osoba1	osoba2
ana123@hotmail.com	zecG@gmail.com
zecG@gmail.com	iva.malic@fer.hr
klaraB@gmail.com	ana123@hotmail.com
zecG@gmail.com	jezV@hotmail.com
Iva.malic@fer.hr	jezV@hotmail.com
ana123@hotmail.com	ana123@hotmail.com

← Uzrokovat će beskonačnu petlju

PostgreSQL

```
WITH RECURSIVE prijatelji
  (prijatelj1, prijatelj2) AS
  ( (SELECT osoba1, osoba2 FROM foaf)
    UNION
    (SELECT prijatelj1, osoba2
      FROM foaf, prijatelji
      WHERE prijatelji.prijatelj2 = foaf.osoba1)
  )
SELECT * FROM prijatelji
WHERE prijatelj1 = 'ana123@hotmail.com'
```

Prepozna beskonačnu rekurziju (zbog UNION, da je korišten UNION ALL rezultat bi bio kao kod SQL Servera) i uspije je razriješiti. Dobije se sljedeći rezultat:

prijatelj1	prijatelj2
ana123@hotmail.com	zecG@gmail.com
ana123@hotmail.com	iva.malic@fer.hr
ana123@hotmail.com	jezV@hotmail.com
ana123@hotmail.com	ana123@hotmail.com

SQL Server

```
WITH prijatelji
  (prijatelj1, prijatelj2) AS
  ( (SELECT osoba1, osoba2 FROM foaf)
    UNION ALL
    (SELECT prijatelj1, osoba2
      FROM foaf, prijatelji
      WHERE prijatelji.prijatelj2 = foaf.osoba1)
  )
SELECT * FROM prijatelji
WHERE prijatelj1 = 'ana123@hotmail.com'
```

Msg 530, Level 16, State 1, Line 1 The statement terminated. The maximum recursion 100 has been exhausted before statement completion.

Rekurzivni upiti u relacijskim sustavima za upravljanje BP

- Microsoft SQL Server 2005 +
- Oracle 11g
- IBM DB2 v2
- PostgreSQL 8.4+
- Firebird 2.1